

第7章 软件工程

软件工程是指应用计算机科学、数学及管理科学等原理，以工程化的原则和方法来解决软件问题的工程，其目的是提高软件生产率、提高软件质量、降低软件成本。IEEE 对软件工程的定义是：将系统的、规范的、可度量的工程化方法应用于软件开发、运行和维护的全过程及上述方法的研究。

软件工程由方法、工具和过程三个部分组成。软件工程方法是完成软件工程项目的手段，它支持整个软件生命周期；软件工程使用的工具是人们在开发软件的活动中智力和体力的扩展与延伸，它自动或半自动地支持软件的开发和管理，支持各种软件文档的生成；软件工程中的过程贯穿软件开发的各个环节，管理人员在软件工程过程中，要对软件开发的质量、进度、成本进行评估、管理和控制，包括人员组织、计划跟踪与控制、成本估算、质量保证和配置管理等。

7.1 软件生命周期

软件产品从形成概念开始，经过开发、使用和维护，直到最后退役的全过程称为软件生命周期或生存周期。一个完整的软件生命周期是以需求为出发点，从提出软件开发计划的那一刻开始，直到软件在实际应用中完全报废为止。软件生命周期的提出是为了更好地管理、维护和升级软件，其中更大的意义在于管理软件开发的步骤和方法。

目前，划分软件生命周期阶段的方法有许多种，软件规模、种类、开发方式和开发环境，以及开发时使用的方法论都影响软件生命周期阶段的划分。对软件生命周期各阶段进行划分，必须遵循一条基本的原则，那就是各阶段的任务应尽可能地相对独立，同一阶段各项任务的性质应尽可能地相同，从而降低每个阶段任务的复杂度，减少不同阶段任务之间的联系，有利于软件工程的组织和管理。

1. 软件生命周期过程

在国家标准《系统与软件工程 软件生存周期过程（GB/T 8566—2022）》中，将软件生存周期中可能执行的活动分为5个基本过程、9个支持过程和7个组织过程。每个生存周期过程划分为一组活动，每一项活动进一步划分为一组任务。

（1）基本过程。基本过程供各主要参与方在软件生存周期期间使用，主要参与方是发起或完成软件产品开发、运行或维护的组织。基本过程分为获取过程、供应过程、开发过程、运作过程和维护过程。获取过程是指为获取系统、软件产品或软件服务的组织（即需方）而定义的活动；供应过程是指为向需方提供系统、软件产品或软件服务的组织（即供方）而定义的活动；开发过程是指为定义并开发软件产品的组织（即开发方）而定义的活动，包括需求分析、设计编码、集成、测试以及与软件产品有关的安装和验收等活动；运作过程是指为在规定的环境中为其用户提供运行计算机系统服务的组织（即操作方）而定义的活动；维护过程是指为提供维

护软件产品服务的组织（即维护方）而定义的活动，也就是对软件的修改进行管理，使它保持合适的运行状态，包括软件产品的迁移和退役。

（2）支持过程。支持过程作为一个有机组成部分支持其他过程，以便取得软件项目的成功，并提高软件项目的质量。支持过程包括文档编制过程、配置管理过程、质量保证过程、验证过程、确认过程、联合评审过程、审核过程、问题解决过程和易用性过程。根据需要，支持过程可被其他过程应用和执行。文档编制过程是指为记录生存周期过程所产生的信息而定义的活动；配置管理过程是指定义配置管理活动；质量保证过程是指为客观地保证软件产品和过程符合规定的需求以及已建立的计划而定义的活动，联合评审、审核、验证和确认可以作为质量保证技术使用；验证过程是指根据软件项目需求，按不同深度为需方、供方或某独立方验证软件产品而定义的活动；确认过程是指为需方、供方或某独立方确认软件项目的软件产品而定义的活动；联合评审过程是指为评价一项活动的状态和产品而定义的活动，该过程可由任何两方应用，其中一方（评审方）以联合讨论会的形式评审另一方（被评审方）；审核过程是指为判定符合需求、计划和合同而定义的活动，该过程可由任何两方应用，其中一方（评审方）审核另一方（被评审方）的软件产品或活动；问题解决过程是指为分析和解决问题（包括不合格）而定义的活动，不论问题的性质或来源如何，它们都是在实施开发、运作、维护或其他过程期间暴露出来的；易用性过程是指为易用性专业人员而定义的活动。

（3）组织过程。组织过程可被某个组织用来建立和实现由相关的生存周期过程和人员组成的基础结构，并不断改进这种结构和过程。应用这些过程通常超出特定的项目和合同的范围，但是，这些特定项目和合同的经验教训有助于改善组织状况。组织过程包括管理过程、基础设施过程、改进过程、人力资源过程、资产管理过程、重用大纲管理过程和领域工程过程。管理过程是指为生存周期中的管理（包括项目管理）而定义的基本活动；基础设施过程是指为建立生存周期过程基础结构而定义的基本活动；改进过程是指为某一组织建立、测量、控制和改进其生存周期过程而定义的需要执行的基本活动；人力资源过程是指为给组织或项目提供拥有技能和知识的员工而定义的活动；资产管理过程是指为组织的资产管理人员而定义的活动；重用大纲管理过程是指为组织的复用大纲主管而定义的活动；领域工程过程是指为领域模型、领域架构的确定及该领域资产的开发和维护而定义的活动。

2. 软件生命周期各阶段的任务

根据国家标准 GB/T 8566—2022，软件生命周期可以划分为可行性研究、需求分析、概要设计、详细设计、实现、组装测试、确认测试、使用、维护、退役 10 个阶段，各自分别对应于软件生存周期的基本过程，如图 7-1 所示。

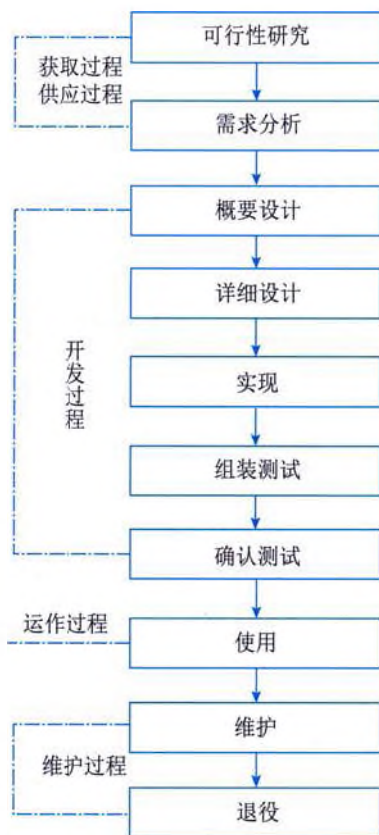


图 7-1 软件生命周期与 5 个基本过程的对应关系

(1) 可行性研究和项目开发计划。通过分析用户提出的软件开发要求，确定软件项目的性质、目标和规模，得出可行性研究报告。如果可行性研究的结果是可行的，就要制定详细的项目开发计划。这两个活动通常被整合在一起进行，在实际工作中通常把它们归类到同一个阶段中。

(2) 需求分析。需求分析工作是软件生命周期中重要的一步，也是决定性的一步。只有通过需求分析，才能把软件功能和性能的总体概念描述为具体的软件需求规格说明，从而奠定软件开发的基础。

(3) 概要设计。根据软件需求规格说明建立软件系统的总体结构和模块间的关系，定义各功能模块接口，设计全局数据库或数据结构，规定设计约束，制定组装测试计划。

(4) 详细设计。将各模块要实现的功能用相应的设计工具详细描述出来。

(5) 实现。写出正确的、易理解的和易维护的程序模块。程序员根据详细设计文档将详细设计转化为程序，完成单元测试。

(6) 组装测试（集成测试）。将经过单元测试的模块逐步进行组装和测试。

(7) 确认测试。测试系统是否达到了系统需求，按照规格说明书的规定，由用户（或在用户积极参与下）对系统进行验收。必要时，还可以再通过现场测试或并行运行等方法对系统进行进一步的测试。

(8) 使用。将软件安装在用户确定的运行环境中，测试通过后移交用户使用。在软件的使用过程中，客户和维护人员必须认真收集发现的软件错误，定期或阶段性地撰写软件问题报告和软件修改报告。

(9) 维护。通过各种必要的维护活动使系统持久地满足用户的需要。

(10) 退役。终止对软件产品的支持，软件停止使用。

7.2 软件开发方法与模型

软件开发方法是指软件开发过程所遵循的办法和步骤，从不同的角度可以对软件开发方法进行不同的分类。从开发风范上看，可分为自顶向下的开发方法与自底向上的开发方法。在实际软件开发中，经常是两种方法结合使用，只不过是应用于开发的不同阶段和以何者为主而已。

软件开发模型则给出了软件开发活动各阶段之间的关系，它是软件开发过程的概括，是软件工程的重要内容。软件开发模型为软件工程管理提供了里程碑和进度表，为软件开发过程提供了原则和方法。

7.2.1 传统软件开发模型

软件开发模型大体上可分为三种类型：第一种是以软件需求完全确定为前提的瀑布模型；第二种是在软件开发初始阶段只能提供基本需求时采用的迭代式或渐进式开发模型，例如，喷泉模型、螺旋模型、统一开发过程和敏捷方法等；第三种是以形式化开发方法为基础的变换模型。

1. 瀑布模型

软件开发生命周期（Software Development Life Cycle, SDLC）模型的发展体现了软件理论的发展。在最早的时候，软件的开发过程处于无序和混乱的状况，人们为了能够控制软件的开发过程，就将软件开发严格区分为多个不同的阶段，并在阶段之间加上严格的审查，这就是瀑布模型产生的起因。

瀑布模型是一种严格定义方法，它将软件开发的过程分为软件计划、需求分析、软件设计、程序编码、软件测试和运行维护 6 个阶段，形如瀑布流水，最终得到软件产品，如图 7-2 所示。

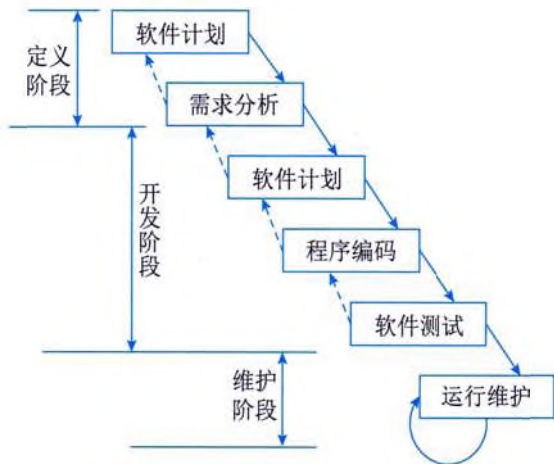


图 7-2 瀑布模型

瀑布模型是一个线性顺序模型，支持线性开发。它假设当线性序列完成之后就能交付一个完善的系统，并没有考虑软件的演化特征。其优点是强调开发的阶段性、早期计划和需求调查以及产品测试，以这样严格的方式构造软件，开发人员很清楚每一步应该做什么，有利于项目管理。

然而，在瀑布模型中，依赖于早期进行的需求调查，不能适应需求的变化；由于是单一流程，开发中的经验教训不能反馈应用于本产品的过程；风险往往推迟至后期的开发阶段才显露出来，从而失去了及早纠正的机会。在瀑布模型中，需求或设计中的错误往往只有到了项目后期才能够被发现，对于项目风险的控制能力较弱，从而导致项目常常延期完成，开发费用超出预算。

2. 演化模型

演化模型主要针对事先不能完整定义需求的软件开发，是在快速开发一个原型的基础上，根据用户在调用原型的过程中提出的反馈意见和建议，对原型进行改进，获得原型的新版本，重复这一过程，直到演化成最终的软件产品。

演化模型的主要优点是，任何功能一经开发就能进入测试，以便验证是否符合产品需求，可以帮助引导出高质量的产品要求。其主要缺点是，如果不加控制地让用户接触开发中尚未稳定的功能，可能对开发人员及用户都会产生负面的影响。

3. 螺旋模型

螺旋模型是瀑布模型与演化模型相结合，并加入两者所忽略的风险分析所建立的一种软件开发模型。螺旋模型是一种演化软件过程模型，它将原型实现的迭代特征与线性顺序模型中的控制和系统化的方面结合起来，使软件的增量版本的快速开发成为可能。在螺旋模型中，软件开发是一系列的增量发布。

螺旋模型沿着螺旋线进行若干次迭代，每次迭代都包括制订计划、风险分析、实施工程和客户评估4个方面的工作。螺旋模型强调风险分析，使得开发人员和用户对每个演化层出现的风险有所了解，继而做出应有的反应。因此，它特别适用于庞大、复杂并具有高风险的系统。

与瀑布模型相比，螺旋模型支持用户需求的动态变化，为用户参与软件开发的所有关键决策提供了方便，有助于提高软件的适应能力，并且为项目管理人员及时调整管理决策提供了便利，从而降低了软件开发的开发风险。在使用螺旋模型进行软件开发时，需要开发人员具有相当丰富的风险评估经验和专门知识。另外，过多的迭代次数会增加开发成本，延迟提交时间。

4. 喷泉模型

喷泉模型是一种以用户需求为动力，以对象为驱动力的模型，主要用于描述面向对象的软件开发过程。该模型认为软件开发过程自下而上的各阶段是相互重叠和多次反复的，就像水喷上去又可以落下来，类似一个喷泉。各个开发阶段没有特定的次序要求，并且可以交互进行，可以在某个开发阶段随时补充其他任何开发阶段中的遗漏。

在喷泉模型中，各活动之间无明显边界，例如，分析和设计之间没有明显的边界。这种特性称为无间隙性。由于对象概念的引入，只用类和关系来表达分析、设计和实现等活动，从而可以较容易地实现活动的迭代和无间隙，提高软件项目开发效率，节省开发时间。

5. 变换模型

变换模型是基于形式化规格说明语言和程序变换的软件开发模型，它对形式化的软件规格说明进行一系列自动或半自动的程序变换，最后映射为计算机能够接受的软件系统。为了确认形式化规格说明与软件需求的一致性，往往以形式化规格说明为基础开发一个软件原型，用户可以从人机界面、系统主要功能和性能等方面对原型进行评审。必要时，可以修改软件需求、形式化规格说明和原型，直至原型被确认为止。这时，开发人员即可对形式化的规格说明进行一系列的程序变换，直至生成计算机可以接受的目标代码。

变换模型的优点是解决了代码结构经多次修改而变坏的问题，减少了许多中间步骤（例如，设计、编码和测试等）。但是，变换模型仍有较大的局限性，以形式化开发方法为基础的变换模型需要严格的数学理论和一整套开发环境的支持。

6. 智能模型

智能模型也称为基于知识的软件开发模型，它综合了上述若干模型，并把专家系统结合在一起。该模型应用基于规则的系统，采用规约和推理机制，帮助开发人员完成开发工作，并使维护在系统规格说明一级进行。为此，需要建立知识库，将模型本身、软件工程知识与特定领域的知识分别存入知识库。

7. V 模型

V 模型是在快速应用开发模型的基础上演变而来的，由于将整个开发过程构造成一个 V 字形而得名。V 模型应用在软件测试方面，和瀑布模型有一些共同的特征。V 模型的过程从左到右，描述了基本的开发过程和测试行为，其价值在于它非常明确地标明了测试过程中存在的不同级别，并且清楚地描述了这些测试阶段和开发过程各阶段的对应关系，如图 7-3 所示。

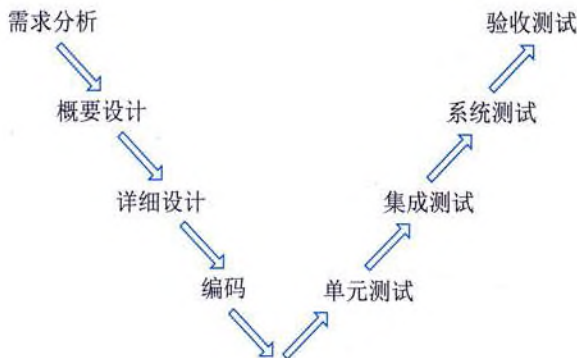


图 7-3 V 模型

在 V 模型中，单元测试是基于代码的测试，最初由开发人员执行，以验证程序代码的各个部分是否已达到预期的功能要求；集成测试验证了两个以上单元之间的集成是否正确，并有针对性地对详细设计中所定义的各单元之间的接口进行检查；在所有单元测试和集成测试完成后，系统测试开始以客户环境模拟系统的运行，验证系统是否达到了在概要设计中所定义的功能和性能；最后，当技术部门完成了所有测试工作后，由业务专家或用户进行验收测试，以确保产品能真正符合用户业务上的需要。

V 模型强调软件开发的协作和速度，将软件实现和验证有机地结合起来，在保证较高的软件质量的情况下缩短开发周期。V 模型适合企业级的软件开发，它更清楚地揭示了软件开发过程的特性及其本质。

7.2.2 快速应用开发

快速应用开发（Rapid Application Development, RAD）是一种比传统生命周期法快得多的开发方法，它强调极短的开发周期。RAD 模型是瀑布模型的一个高速变种，通过使用基于构件的开发方法获得快速开发。如果需求理解得很好，且约束了项目范围，利用这种模型可以很快开发出功能完善的信息系统。

1. RAD 的基本思想

RAD 的基本思想体现在以下 4 个方面：

- (1) 让用户更主动地参与到系统分析、设计和构造活动中来。
- (2) 将项目开发组织成一系列重点突出的研讨会，研讨会要让项目投资方、用户、系统分析师、设计人员和开发人员一起参与。
- (3) 通过一种迭代的构造方法，加速需求分析和设计阶段。

(4) 让用户提前看到一个可工作的系统。

2. RAD 的开发阶段

RAD 的流程从业务建模开始，随后是数据建模、过程建模、应用生成、测试与交付。

(1) 业务建模。确定驱动业务过程运作的信息、要生成的信息、如何生成、信息流的去向及其处理等，可以使用数据流图来帮助建立业务模型。

(2) 数据建模。为支持业务过程的数据流查找数据对象集合、定义数据对象属性，并与其他数据对象的关系构成数据模型，可以使用 E-R 图来帮助建立数据模型。

(3) 过程建模。将数据对象变换为要完成一个业务功能所需的信息流，创建过程以描述增加、修改、删除或获取某个数据对象，即细化数据流图中的加工。

(4) 应用生成。利用第四代语言(4GL)写出处理程序，复用已有构件或创建新的可复用构件，利用环境提供的工具自动生成并构造出整个应用系统。

(5) 测试与交付。因为 RAD 强调复用，许多构件已经是测试过的，这就减少了测试的时间。由于大量复用，所以一般只做总体测试，但新创建的构件还是要测试的。

3. RAD 的特点

RAD 采用基于构件的开发方法，复用已有的程序结构（如果可能的话）或使用构件，或者创建可复用的构件（如果需要的话）。在所有情况下，均可以使用 CASE 工具辅助进行软件构建。如果一个业务能够被模块化，使得其中每一个主要功能均可以在不到三个月的时间内完成，那么，它就是 RAD 的一个候选者。每个主要功能可由一个单独的 RAD 组来实现，最后再集成起来，形成一个整体。

RAD 通过大量使用可复用构件，加快了开发速度。但是，RAD 也具有以下局限性：

(1) 并非所有应用都适合 RAD。RAD 对模块化要求比较高，如果有哪一项功能不能被模块化，那么 RAD 所需要的构件就会有问题；如果高性能是一个指标，且该指标必须通过调整接口使其适应系统构件才能获得，则 RAD 也有可能不能奏效。

(2) 开发者和客户必须在很短的时间完成一系列的需求分析，任何一方配合不当，都会导致 RAD 项目失败。

(3) RAD 只能用于管理信息系统的开发，不适合技术风险很高的情况。例如，当一个新系统要采用很多新技术，或当新系统要与现有系统有较高的互操作性时，就不适合使用 RAD。

7.2.3 统一过程模型

统一过程（Unified Process, UP）是一个通用过程框架，可以用于种类广泛的软件系统、不同的应用领域、不同的组织类型、不同的性能水平和不同的项目规模。UP 是基于构件的，在为软件系统建模时，UP 使用的是 UML。与其他软件过程相比，UP 具有三个显著的特点，即用例驱动、以架构为中心、迭代和增量。

UP 提供了在开发组织中分派任务和责任的纪律化方法，它的目标是在可预见的日程和预算前提下，确保满足最终用户需求的高质量产品。对所有的关键开发活动，它为每个团队成员提供了使用准则、模板和工具指导。而通过对相同基础知识的一致理解，在进行需求分析、设计、

测试或配置管理等工作时, 均能确保全体成员共享相同的知识、过程和开发软件的视图。

1. RUP 概述

RUP (Rational Unified Process) 是 Rational 公司开发和维护的过程产品, 是由 Objectory 过程演化而来的。RUP 将项目管理、业务建模、分析与设计等统一起来, 贯穿整个开发过程。RUP 采用 Internet 技术, 可以增强团队的开发效率, 并为所有成员提供最佳的软件实现方案, 它使团队中每个开发人员的见解和思想得到统一, 使开发小组成员的沟通更为容易, 而这正是任何项目取得成功的关键因素。RUP 可以增强开发人员对软件的预见性, 最终的好处就是提高了软件质量, 并有效缩短了软件从开发到投放市场的时间。RUP 为软件开发提供了规范性的指南、模板和范例, 可用来开发所有类型的应用。

RUP 中的软件过程在时间上被分解为 4 个顺序的阶段, 分别是初始阶段、细化阶段、构建阶段和移交阶段。每个阶段结束时都要安排一次技术评审, 以确定这个阶段的目标是否已经满足。如果评审结果令人满意, 就允许项目进入下一个阶段。基于 RUP 的软件过程模型如图 7-4 所示。

从图 7-4 中可以看出, 基于 RUP 的软件过程是一个迭代过程。初始、细化、构建和移交 4 个阶段就是一个开发周期, 每次经过这 4 个阶段就会产生一代软件。除非产品退役, 否则, 通过重复同样的 4 个阶段, 产品将演化为下一代产品, 但每一次的侧重点都将放在不同的阶段上。

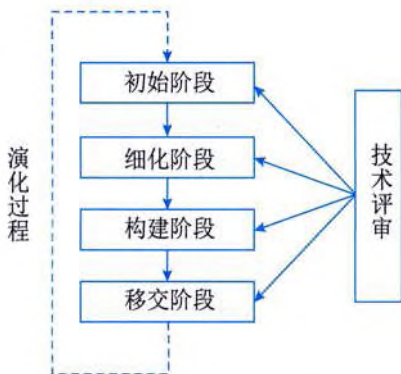


图 7-4 基于 RUP 的软件过程

2. 初始阶段

初始阶段的任务是为系统建立业务模型并确定项目的边界。在初始阶段, 必须识别所有与系统交互的外部实体, 定义系统与外部实体交互的特性。在这个阶段关注的是整个项目的业务和需求方面的主要风险。对于建立在原有系统基础上的开发项目来说, 初始阶段可能很短。初始阶段的实现过程如下:

(1) 明确项目规模。建立项目的软件规模和边界条件, 包括验收标准; 了解环境及重要的需求和约束, 识别系统的关键用例。

(2) 评估项目风险。软件过程主要关心的是软件开发的已知方面, 只能准确描述、计划、分配和评审那些已经知道将要完成的事情。风险管理则主要关心未知方面。在基于 RUP 的迭代式软件过程中, 很多决策要受风险决定。要达到这个目的, 开发人员需要详细了解项目所面临的风险, 并对如何降低或处理风险有明确的策略。

(3) 制订项目计划。估计整个项目的总体成本、进度和人员配备。综合考虑备选架构, 评估设计和自制/外购/复用方面的方案, 从而估算出成本、进度和资源。在这个过程中, 要通过对一些概念的证实来证明可行性, 可以采用可模拟需求的模型形式或用于探索高风险区的初始原型。初始阶段的原型设计工作应该限制在确信解决方案可行就可以了, 具体实现留到细化阶段和构建阶段。

(4) 阶段技术评审。初始阶段结束时要进行一次技术评审，检查初始阶段的目标是否完成，并决定继续进行项目还是取消项目。在评审过程中，需要考虑项目的规模定义、成本和进度估算是否适中、估算根据是否可靠、需求是否正确、开发方和用户方对软件需求的理解是否达成一致、是否已经确定所有风险且有针对每个风险的规避策略等问题。

3. 细化阶段

细化阶段的任务是分析问题领域，建立完善的架构，淘汰项目中最高风险的元素。在细化阶段，必须在理解整个系统的基础上，对架构做出决策，包括其范围、主要功能和诸如性能等非功能需求，同时为项目建立支持环境。细化阶段的实现过程如下：

(1) 确定架构。确保架构、需求和计划足够稳定，充分减少风险，从而能够有预见性地确定开发所需的成本和开发进度。通过处理架构中的关键场景，建立一个已确定基线的架构，并验证其将在适当时间、以合理的成本支持系统需求。

(2) 制订构建阶段计划。为构建阶段制订详细的过程计划，并为其建立基线。

(3) 建立支持环境。包括开发环境、开发流程、支持构建团队所需的工具和自动化 / 半自动化支持。

(4) 选择构件。评估现有的构件库和潜在构件，充分了解自制 / 外购 / 复用决策，以便有把握地确定构建阶段的成本和进度。集成所选构件，并按主要场景进行评估。

(5) 阶段技术评审。评审时，需要检验详细的系统目标和范围、架构的选择，以及主要风险的解决方案。

在细化阶段，可执行的原型依赖于项目的范围、规模、风险和先进程度。必须至少处理初始阶段中识别的关键用例，因为关键用例通常揭示了项目的主要技术风险。

4. 构建阶段

在构建阶段，要开发所有剩余的构件和应用程序功能，把这些构件集成为产品，并进行详细测试。从某种意义上说，构建阶段是一个制造过程，其重点放在管理资源及控制操作，以优化成本、进度和质量。构建阶段的主要任务是通过优化资源和避免不必要的报废和返工，使开发成本降到最低；完成所有所需功能的分析、开发和测试，快速完成可用的版本；确定软件、场地和用户是否已经为部署软件做好准备。

在构建阶段，开发团队的工作可以实现某种程度的并行。一些项目的规模大得足够产生许多并行的增量构建过程，即使是较小的项目，也通常包括可以相互独立开发的构件，从而使各团队之间实现并行开发。这些并行活动在加速版本发布的有效性的同时，也增加了资源管理和 workflow 同步的复杂性。

构建阶段结束时也要进行技术评审，评审产品是否可以在 β 测试环境中进行安装和运行。

5. 移交阶段

当基线已经足够完善，可以安装到最终用户实际环境中时，则进入移交阶段。移交阶段的重点是确保软件对最终用户是可用的。移交阶段的主要任务是进行 β 测试，制作产品发布版本；对最终用户支持文档定稿；按用户的需求确认新系统；培训用户和维护人员；获得用户对

当前版本的反馈,基于反馈调整产品,例如,进行调试、性能或可用性的增强等。

移交阶段结束时也要进行技术评审,评审目标是否实现、是否应该开始演化过程、用户对交付的产品是否满意等。

从本节的介绍可以看出,RUP 由于太过庞大和复杂,相对于轻量级的敏捷方法来说,显得死板和难以实施。RUP 不但不能快速适应需求的变化,而且变更一个需求要经历复杂的过程和很多额外的工作。对于较小的组织和项目来说,使用敏捷方法可能比较合适,而使用 RUP 似乎有些费力不讨好。

7.2.4 敏捷方法

敏捷方法是从 20 世纪 90 年代开始引起广泛关注的一些新型软件开发方法,以应对快速变化的需求。虽然它们的具体名称、理念、过程、术语都不尽相同,但相对于“非敏捷”而言,它们更强调开发团队与用户之间的紧密协作、面对面的沟通、频繁交付新的软件版本、紧凑而自我组织型的团队等,也更注重人的作用。

1. 敏捷宣言

2001 年,Kent Beck 等人组织了敏捷联盟,阐述了敏捷开发的原则,试图强调灵活性在快速且有效地开发软件中所发挥的作用,他们共同签署了敏捷软件开发宣言,该宣言认为,个体和交互胜过过程和工具;可工作的软件胜过大量的文档;客户合作胜过合同谈判;响应变化胜过遵循计划。

敏捷方法强调:让客户满意和软件尽早增量发布;小而高度自主的项目团队;非正式的方法;最小化软件工作产品以及整体精简开发。产生这种情况的原因是,在绝大多数软件开发过程中,提前预测哪些需求是稳定的和哪些需求会变化非常困难;对于软件项目构建来说,设计和实现是交错的;从制定计划的角度来看,分析、设计、实现和测试并不容易预测;可执行原型和部分实现的可运行系统是了解用户需求和反馈的有效媒介。

目前,主要的敏捷方法有极限编程(eXtreme Programming, XP)、自适应软件开发(Adaptive Software Development, ASD)、水晶方法(Crystal)、特性驱动开发(Feature-Driven Development, FDD)、动态系统开发方法(Dynamic Systems Development Method, DSDM)、测试驱动开发(Test-Driven Development, TDD)、Scrum、敏捷数据库技术(Agile Database Techniques, AD)和精益软件开发(Lean Software Development)等。虽然这些过程模型在实践上有差异,但都遵循敏捷宣言或者敏捷联盟所定义的基本原则。这些原则包括客户参与、增量式移交、简单性、接受变更、强调开发人员的作用和及时反馈等。

2. 敏捷方法的特点

敏捷方法是一种以人为核心、迭代、循序渐进的开发方法。在敏捷方法中,软件项目的构建被切分成多个子项目,各个子项目的成果都经过测试,具备集成和可运行的特征。在敏捷方法中,从开发者的角度来看,主要的关注点有短平快的会议、小版本发布、较少的文档、合作为重、客户直接参与、自动化测试、适应性计划调整和结对编程;从管理者的角度来看,主要的关注点有测试驱动开发、持续集成和重构。

近年来，虽然敏捷方法发展得较快，但在实施的过程中，也暴露出很多问题，一些敏捷方法的基本原则很难实施，主要体现在以下4个方面：

- (1) 客户参与往往依赖于客户参与的意愿和客户自身的代表性。
- (2) 团队成员的性格可能不适合激烈的投入，可能无法做到与其他成员之间的美好沟通。
- (3) 对系统的变更做出优先级排序可能是极端困难的。

(4) 维护系统的简洁性往往需要额外的工作，但迫于时间表的压力，可能没有时间执行系统简化过程。

与 RUP 相比，敏捷方法的周期可能更短。敏捷方法在几周或几个月的时间内完成相对较小的功能，强调的是能尽早将尽量小的可用的功能交付使用，并在整个项目周期中持续改善和增强，并且更加强调团队中的高度协作。相对而言，敏捷方法主要适用于以下场合：

(1) 项目团队的人数不能太多，适合于规模较小的项目。

(2) 项目经常发生变更。敏捷方法适用于需求萌动并且快速改变的情况，如果系统有比较高的关键性、可靠性、安全性方面的要求，则可能不完全适合。

(3) 高风险项目的实施。

(4) 从组织结构的角度看，组织结构的文化、人员、沟通性决定了敏捷方法是否适用。与这些相关联的关键成功因素有组织文化必须支持谈判、人员彼此信任、人少但是精干、开发人员所做的决定得到认可、环境设施满足团队成员之间快速沟通的需要。

3. 极限编程方法

敏捷方法中最著名的就是极限编程（XP），XP 是一种轻量、高效、低风险、柔性、可预测、科学且充满乐趣的软件开发方式，适用于小型或中型软件开发团队，并且客户的需求模糊或需求多变。与其他方法相比，XP 方法最大的不同如下：

(1) 在更短的周期内，更早地提供具体、持续的反馈信息。

(2) 迭代地进行计划编制，首先在最开始迅速生成一个总体计划，然后在整个项目开发过程中不断地发展它。

(3) 依赖于自动测试程序来监控开发进度，并尽早捕获缺陷。

(4) 依赖于口头交流、测试和源程序进行沟通。

(5) 倡导持续的演化式的设计。

(6) 依赖于开发团队内部的紧密协作。

(7) 尽可能达到程序员短期利益和项目长期利益的平衡。

XP 由价值观、原则、实践和行为 4 个部分组成，它们彼此相互依赖、关联，并通过行为贯穿于整个生命周期。XP 的核心是其总结的四大价值观，即沟通、简单、反馈和勇气。它们是 XP 的基础，也是 XP 的灵魂。XP 的 5 个原则是快速反馈、简单性假设、逐步修改、提倡更改和优质工作。而在 XP 方法中，贯彻的是“小步快走”的开发原则，因此工作质量决不可打折扣，通常采用测试先行的编码方式来提供支持。

4. 动态系统开发方法

动态系统开发方法（DSDM）倡导以业务为核心，快速而有效地进行系统开发。可以把

DSDM 看成一种控制框架，其重点在于快速交付并补充如何应用这些控制的指导原则。

DSDM 是一整套的方法论，不仅仅包括软件开发内容和实践，也包括了组织结构、项目管理、估算、工具环境、测试、配置管理、风险管理、重用等各个方面的内容。

1) DSDM 的基本观点

DSDM 认为任何事情都不可能一次性圆满完成，应该用 20% 的时间完成 80% 的有用功能，以适合商业目的为准。实施的思路是：在时间进度和可用资源预先固定的情况下，力争最大化地满足业务需求（传统方法一般是需求固定，时间和资源可变），交付所需要的系统。对于交付的系统，必须达到足够的稳定程度，以在实际环境中运行；对于业务方面的某些紧急需求，也必须能够在短时间内得到满足，并在后续迭代阶段对功能进行完善。

2) DSDM 的基本原则

DSDM 的基本原则包括：

- 用户必须参与。
- 必须授权 DSDM 团队进行决策。
- 注重频繁交付产品。
- 判断产品是否可接受的基本标准是是否符合业务目的。
- 对准确的业务解决方案需要采用循环和增量开发。
- 开发期间的所有更改都是可逆的。
- 在高层次上制定需求的基线（以在低优先级的项目上获得一定的灵活性）。
- 在整个生命周期集成测试。
- 在所有参与者之间采用协作和合作方法。

5. Scrum

Scrum 是一个用于开发和维护复杂产品的框架，是一个增量的、迭代的开发过程。在这个框架中，整个开发过程由若干个短的迭代周期组成，一个短的迭代周期称为一个 Sprint，每个 Sprint 的建议长度是 2 ~ 4 周。

在 Scrum 中，使用产品 Backlog 来管理产品的需求，产品 Backlog 是一个按照商业价值排序的需求列表，列表条目的体现形式通常为用户故事。Scrum 团队总是先开发对客户具有较高价值的需求。在 Sprint 中，Scrum 团队从产品 Backlog 中挑选最高优先级的需求进行开发。

挑选的需求在 Sprint 计划会议上经过讨论、分析和估算得到相应的任务列表，称为 Sprint Backlog。在每个迭代结束时，Scrum 团队将递交潜在可交付的产品增量。Scrum 起源于软件开发项目，但它适用于任何复杂的或创新性的项目。

7.3 软件开发环境与工具

软件开发环境（Software Development Environment, SDE）是指支持软件的工程化开发和维护而使用的一组软件，由软件工具集和环境集成机制构成。软件工具是指 CASE 工具，用以

支持软件开发的相关过程、活动和任务；环境集成机制是指为工具集成和软件开发、维护及管理提供统一的支持。通过环境集成机制，各工具用统一的数据接口规范存储或访问环境信息库，采用统一的界面形式，保证界面的一致性，同时，为各工具或开发活动之间的通信、切换、调度和协同工作提供支持。

7.3.1 软件开发环境

软件开发环境应支持多种集成机制，例如，平台集成、数据集成、界面集成、控制集成和过程集成等。软件开发环境应支持小组工作方式，并为其提供配置管理，环境的服务可用于支持各种软件开发活动，包括分析、设计、编程、调试和文档等。

较完善的软件开发环境通常具有多种功能，例如，软件开发的一致性与完整性维护，配置管理及版本控制，数据的多种表示形式及其在不同形式之间的自动转换，信息的自动检索与更新，项目控制和管理，以及对开发方法学的支持。软件开发环境具有集成性、开放性、可裁减性、数据格式一致性、风格统一的用户界面等特性，因而能大幅度提高软件生产率。

1. 软件开发环境的分类

软件开发环境可按以下几种角度进行分类：

（1）按软件开发模型与开发方法分类，有支持瀑布模型、演化模型、螺旋模型和喷泉模型等不同模型，以及结构化方法、面向对象方法等不同方法的软件开发环境。

（2）按功能与结构特点分类，有单体型、协同型、分散型和并发型等多种类型的软件开发环境。

（3）按应用范围分类，有通用型和专用型软件开发环境。其中，专用型软件开发环境与应用领域有关。

（4）按开发阶段分类，有前端开发环境（支持系统规划、分析、设计等阶段的活动）、后端开发环境（支持编程、测试等阶段的活动）、软件维护环境和逆向工程环境等。

2. 集成机制

集成机制根据功能的不同，可划分为环境信息库、过程控制与消息服务器、环境用户界面三个部分。

（1）环境信息库。环境信息库是软件开发环境的核心，用以存储与系统开发有关的信息，并支持信息的交流与共享。环境信息库中主要存储两类信息：一类是开发过程中产生的有关被开发系统的信息，例如，分析文档、设计文档和测试报告等；另一类是环境提供的支持信息，例如，文档模板、系统配置、过程模型和可复用构件等。

（2）过程控制与消息服务器。过程控制与消息服务器是实现过程集成和控制集成的基础。过程集成是按照具体软件开发过程的要求进行工具的选择与组合，控制集成使各工具之间进行并行通信和协同工作。

（3）环境用户界面。环境用户界面包括环境总界面和由它实行统一控制的各环境部件及工具的界面。统一的、具有一致性的用户界面是软件开发环境的重要特征，是充分发挥环境的优越性、高效地使用工具并减轻用户的学习负担的保证。

3. 集成计算机辅助软件工程

目前,随着软件开发工具的积累与自动化工具的增多,软件开发环境已经进入了第三代,即集成计算机辅助软件工程(Integrated Computer-Aided Software Engineering, ICASE)阶段。集成方式经历了从点到点的数据转换(早期CASE采用的集成方式),到公共用户界面(第二代CASE,在一致的界面下调用众多不同的工具),再到目前的信息库方式。这是ICASE的主要集成方式。ICASE不仅提供数据集成和控制集成,还提供了一组用户界面管理设施和一大批工具,包括垂直工具集(支持软件生命周期各阶段,保证生成信息的完备性和一致性)、水平工具集(用于不同的软件开发方法)和开放工具槽(用于连接新的工具)。

ICASE的信息库不仅定义了面向对象的数据库管理系统,提供了数据-数据集成机制,还建立了可以被环境中所有工具访问的数据模型,提供了数据-工具集成机制,实现了配置管理功能。ICASE的进一步发展则是与软件开发方法的结合,以及智能化的ICASE。

ICASE的最终目标是实现应用软件的全自动开发,即开发人员只要写好软件的需求规格说明书,ICASE就能自动完成软件开发工作,即自动生成供用户直接使用的软件和有关文档。

7.3.2 软件开发工具

软件开发环境中的工具可包括支持特定过程模型和开发方法的工具(例如,支持瀑布模型及数据流方法的工具,支持面向对象方法的工具等)和独立于模型和方法的工具(例如,界面辅助生成工具和文档出版工具等),也可包括管理类工具和针对特定领域的应用类工具。所有这些工具可分为贯穿整个开发过程的工具(例如,项目管理工具)和解决软件生命周期中某一阶段问题的工具(例如,软件估算工具等)。

1. 软件工具的分类

在软件生命周期中,要使用很多软件工具,从其功能上进行划分,可以分为软件开发工具、软件维护工具、软件管理和支持工具三类。

(1) 软件开发工具。软件开发工具用来辅助开发人员进行软件开发活动,包括需求分析工具、设计工具、编码与排错工具等。

(2) 软件维护工具。软件维护工具用来辅助维护人员对软件代码及其文档进行各种维护活动,包括版本管理工具、文档分析工具、开发信息库工具、逆向工程工具和再工程工具等。

(3) 软件管理和支持工具。软件管理和支持工具用来辅助管理人员和软件支持人员的管理活动和支持活动,以确保软件高质量完成,包括项目管理工具、配置管理工具和软件评价工具等。

2. 开发工具的选择

开发工具的选择是软件项目成功的要素之一。对软件开发工具的评价和选择,要参照具体软件项目对开发工具的选择标准和要求,从功能、易用性、稳健性、硬件要求和性能,以及服务和支持等方面来衡量。简单地说,开发工具的选择主要决定于两个因素,分别是所开发系统的最终用户和开发人员。最终用户需求是一切软件的来源和归宿,也是影响开发工具的决定性因素;开发人员的爱好、习惯和经验也影响着开发工具的选择。

需要强调的是，开发工具的比较没有绝对的标准。评价一种开发工具，不仅要看它对设计模式和对象结构，以及管理的支撑情况，更重要的是要针对具体的使用环境、开发方法、软件架构和开发人员，以及最终用户来评价一种工具的适宜程度。

3. 快速开发工具

在 RAD 方法中，所包括的工具主要有数据库编程语言、界面生成器和报告生成器等。RAD 工具主要使用可视化技术。可视化技术是一种通过集成细粒度可复用构件来构造软件的方法，其主要思想是用图形工具和可复用构件来交互地编制程序。常用的 RAD 工具有 Microsoft Visual Studio.NET、Sun Java Studio Creator、WebLogic Workshop 和 Borland C# Builder 等。一般的可视化编程工具还有应用向导提供模板，按照步骤对程序员进行交互式指导，让用户定制自己的应用，然后就可以生成应用程序的框架代码，用户再在适当的地方添加或修改，以适应自己的需求。

7.4 软件过程管理

在无规则和混乱的管理条件下，先进的软件开发技术和工具并不能发挥应有的作用。于是，人们认识到，改进软件过程的管理是解决上述难题的突破口。但是，各个软件组织的过程成熟度有着较大的差别。为了做出客观、公正的比较，就需要建立一种衡量的标准。使用此标准一方面可以评价软件开发方的质量保证能力，在软件项目评标活动中选择开发方；另一方面，该标准也必然成为软件组织加强质量管理和提高软件产品质量的依据。

软件过程是软件生命周期中的一系列相关活动，即用于开发和维护软件及相关产品的一系列活动。软件产品的质量取决于软件过程，具有良好软件过程的组织能够开发出高质量的软件产品。

7.4.1 软件能力成熟度模型

软件能力成熟度模型（Capability Maturity Model, CMM）是一个概念模型，模型框架和表示是刚性的，不能随意改变，但模型的解释和实现有一定弹性。CMM 提供了一个软件能力成熟度的框架，它将软件过程改进的步骤组织成 5 个成熟度等级，为软件过程不断改进奠定了一个循序渐进的基础。

1. CMM 的等级

CMM 的目的是帮助组织对软件过程进行管理和改进，增强开发与改进能力，从而能按时地、不超预算地开发出高质量的软件。CMM 的 5 个成熟度等级分别为初始级、可重复级、已定义级、可管理级和优化级。

（1）初始级。初始级是未加定义的随意过程，软件过程的特点是无秩序的，有时甚至是混乱的。软件过程定义几乎处于无章法、无步骤可循的状态，软件产品所取得的成功往往依赖于极个别别人的努力和机遇。

（2）可重复级。可重复级是规则化和纪律化的过程，软件过程已建立了基本的项目管理流

程,可用于对成本、进度和功能特性进行跟踪。对类似的应用项目有章可循,并能重复以往所取得的成功。

(3) 已定义级。已定义级是标准化和一致化的过程,用于管理和工程活动的软件过程均已文档化、标准化,并形成了整个软件组织的标准软件过程。全部项目均采用与实际情况相吻合的、适当修改后的标准软件过程来进行操作。

(4) 可管理级。已管理级是可预测的过程,软件过程和产品质量有详细的度量标准。软件过程和产品质量得到了定量的认识和控制。

(5) 优化级。优化级是持续改进的过程,通过对来自过程、新概念和新技术等方面的各种有用信息的定量分析,能够不断地、持续性地对过程进行改进。

2. 关键过程域

除初始级以外,CMM 的每个级别的实现都定义成可操作的,每一级包含了实现这一级目标的若干关键过程域(Key Process Area, KPA),如表 7-1 所示。

表 7-1 关键过程域的分类

过程分类 成熟度等级	管理方面	组织方面	工程方面
优化级		技术变更管理 过程变更管理	缺陷预防
可管理级	量化过程管理		软件质量管理
已定义级	集成软件管理 组间合作	组织过程焦点 组织过程定义 培训计划	软件产品工程 同行评审
可重复级	需求管理 软件项目计划 软件项目跟踪与监控 软件子合同管理 软件质量保证 软件配置管理		

每个 KPA 都是由关键实施活动(Key Practices, KP)组成的,它们的执行表明该 KPA 在一个组织内部得到实现。

3. 能力成熟度模型集成

能力成熟度模型集成(Capability Maturity Model Integration, CMMI)融合了多种模型,形成了组织范围内过程改进的单一集成模型,其主要目的是消除不同模型之间的不一致和重复,降低基于模型进行改进的成本。CMMI 继承了 CMM 的阶段表示法和 EIA/IS 731 的连续式表示法。这两种表示方法各有优缺点,均采用统一的 24 个过程域,它们在逻辑上是等价的,对同一个组织采用两种模型分别进行 CMMI 评估,得到的结论应该是相同的。

(1) 阶段式模型。阶段式模型基本沿袭 CMM 模型框架,仍保持 5 个成熟度等级,但关键

过程域做了一些调整和扩充，如表 7-2 所示。

表 7-2 阶段式模型的过程域分组

成熟度等级	过程域
可重复级	需求管理、项目计划、配置管理、项目监督与控制、供应商合同管理、度量和分析、过程和产品质量保证
已定义级	需求开发、技术解决方案、产品集成、验证、确认、组织级过程焦点、组织级过程定义、组织级培训、集成项目管理、风险管理、集成化的团队、决策分析和解决方案、组织级集成环境
可管理级	组织级过程性能、定量项目管理
优化级	组织级改革与实施、因果分析和解决方案

当组织通过了某一等级过程域中的全部过程，即意味着该组织的成熟度达到了这一等级。利用阶段式模型对组织进行成熟度度量，概念清晰、易于理解、便于操作。

（2）连续式模型。与阶段式模型相比，连续式模型没有与组织成熟度相关的几个阶段。连续式模型将 24 个过程域按照功能划分为过程管理、项目管理、工程和支持 4 个过程组。每组包含的过程域如表 7-3 所示。

表 7-3 连续式模型的过程域分组

过程组	过程域
过程管理	组织级过程焦点、组织级过程定义、组织级培训、组织级过程性能、组织级改革与实施
项目管理	项目计划、项目监督与控制、供应商合同管理、集成项目管理、风险管理、集成化的团队、定量项目管理
工程	需求管理、需求开发、技术解决方案、产品集成、验证、确认
支持	配置管理、度量和分析、过程和产品质量保证、决策分析和解决方案、组织级集成环境、因果分析和解决方案

连续式模型的过程域强调实践，每个过程域代表组织某一方面的能力。每个过程域的能力均分为 5 级，所有过程域共同的能力等级决定组织的能力等级。连续式模型允许组织对过程域进行裁剪，也允许对不同的过程域采用不同的能力等级。采用这种模式的评估结果用能力特征

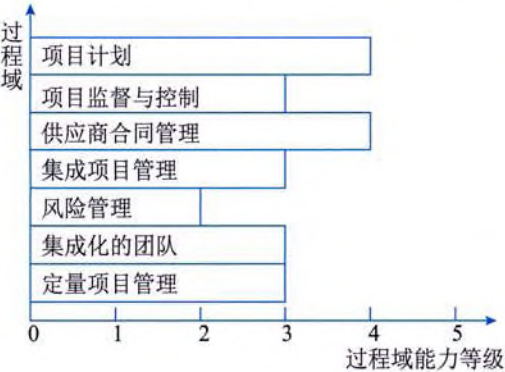


图 7-5 能力特征图示例

图表示，如图 7-5 所示。连续式模型允许一个过程域出现在多个特征图中，这些特征图分别代表某种能力的过程域的子集。

CMM 和 CMMI 是提高软件组织的成熟度和软件过程能力的有效模型和工具，组织无论采用 CMM 模型还是采用 CMMI 模型，无论是使用阶段式模型还是使用连续式模型，都能提高软件过程的成熟度，都能提高项目的软件过程能力，用两种模型或两种方法评价的结论应该是基本一致的。

7.4.2 软件过程评估

软件过程评估是根据过程模型或其他模型对组织的软件过程所进行的规范的评估。软件过程评估是由接受过培训的专业软件人员所组成的小组对组织的当前软件过程进行评估,以确定其状态,确定组织所面临的与软件过程相关事务的优先级,并从组织中获得对软件过程改进的支持。

1. CMM 模型

CMM 的一个重要思想是帮助软件组织通过基于模型的过程改进,达到使其软件过程向更高的成熟度等级迈进的目标。在这个过程中,一个组织必须建立起自己的软件过程,并根据 CMM 模型的要求对过程进行评估。根据评估的结果来进一步改进自己的软件过程,然后再一次评估,以期达到更高的成熟度等级,或者防止软件过程成熟度能力退化。如此反复循环,最终使一个组织的软件过程能力趋于更加成熟。基于 CMM 的评估方法分为以下 6 个步骤:

(1) 成立评估小组,小组由软件工程和管理工作经验丰富的专家组成,小组成员应接受过 CMM 基本概念和评估方法的专门培训。

(2) 参评单位的代表认真填写成熟度问卷表,并回答有关问题。

(3) 评估小组分析调查问卷。

(4) 评估小组现场访问、召开座谈会、审核过程文档,判断 KPA 的实践活动是否达到预定目标,并将结论记入文档。

(5) 整理调查结果,撰写调查报告,指明软件过程的强项和弱项。

(6) 绘制 KPA 剖面图,显示是否达到 KPA 的目标,并向有关部门提交评估的结论性意见。

2. Trillum 模型

Trillum 模型是一个主要用于嵌入式软件开发和支持的能力评估模型,它以 CMM 模型为基础,同时有新的发展。Trillum 模型的主要特点如下:

(1) 模型的架构建立在路线图的基础上。

(2) 模型不仅适用于软件,同时适用于硬件。

(3) 模型强调以用户为关注的焦点。

(4) 模型包括技术成熟度,主要面向通信产品。

3. Bootstrap 方法

Bootstrap 参考 ISO 9000-3 和 ESA PSS-05 等软件标准,设计了非常详细的过程质量属性结构,包括组织资源管理、测试方法和生命周期技术等 17 个属性,改进了 CMM 的问卷表和成熟度计算方法,使其可用于过程的每一个质量属性,从而得到一个过程质量剖面。Bootstrap 可适用于各类软件组织,在欧洲有很大的影响力。

4. ISO/IEC 15504 标准

ISO/IEC 15504 分为 9 个部分,分别是概念与介绍指南、过程与过程能力的参考模型、实施评估、评估实施指南、一个评估模型和指示指南、评估员资格认证指南、过程改进应用指南、

判断供应商过程能力指南和词汇表，其中第一部分是资料，第二部分和第三部分是标准，其他部分都是参考性的。在 ISO/IEC 15504 的第二部分（过程与过程能力的参考模型）中，在比较高的层次上详细定义了一个用于过程评估的二维参考模型，即过程维和能力维。通过将过程中的特点与不同的能力等级相比较，可以用此参考模型定义的一系列过程和框架对过程能力加以评估。

在 ISO/IEC 15504 中，能力等级是针对每个过程的，它定义了 6 级过程性能，每一级都用主要的过程特征术语和用于性能测量的属性来描述。具体包括：不完善的过程、已实施的过程、已计划与已跟踪的过程、已建立的过程、可预测的过程、优化的过程。

5. SJ/T 11234—2001 标准

我国行业标准《软件过程能力评估模型（SJ/T 11234—2001）》针对软件组织对自身软件过程能力进行内部改进的需要，与 CMMI 连续式模型基本相同。该模型有 22 个过程域，分为四大类，分别是过程管理类、项目管理类、工程类和支持类，如表 7-4 所示。

表 7-4 SJ/T 11234—2001 标准的过程域分组

过程域分组	过程域
过程管理	组织级过程焦点、组织级过程定义、组织级培训、组织级过程性能、组织级改革与实施
项目管理	项目计划、项目监督与控制、供方协议管理、集成项目管理、风险管理、定量项目管理
工程	需求管理、需求开发、技术解决方案、产品集成、验证、确认
支持	配置管理、度量和分析、过程和产品质量保证、决策分析和解决方案、因果分析和解决方案

SJ/T 11234—2001 的每个过程能力划分为 6 个评估等级：不完整级、已执行级、受管理级、已定义级、定量管理级和持续优化级。每个等级包含了通用目标、通用惯例、特定目标和特定惯例，它们组成一套衡量准则。不完整级是反映那些没有得到完整执行过程的状态，可能实现了部分特定目标，也可能什么目标都没有实现；处于已执行级的过程实现了全部特定目标；受管理级、已定义级、定量管理级和持续优化级不仅实现了全部特定目标，而且依次实现了对应更高的通用目标。

7.5 软件重用和再工程

软件重用是在两次或多次不同的软件开发过程中重复使用相同或相似软件元素的过程。软件元素包括程序代码、测试用例、设计文档、设计过程、需求分析文档甚至领域知识。通常，可重用的元素也称作软构件，可重用的软构件越大，重用的粒度越大。软件再工程指通过对目标系统的检查和改造，使用逆向工程、重构和正向工程方法，将现存系统重新构造为新的形式，开发出质量更高、维护性更好的软件。

7.5.1 可重用的软件构件

软件重用是在软件开发过程中重复使用相同或相似的软件元素的过程。这些软件元素包括应用领域知识、开发经验、设计经验、体系结构、需求分析文档、设计文档、程序代码及测试用例等。对于新的软件开发项目而言,它们是构成整个软件系统的部件,或者在软件开发过程中可发挥某种作用。通常把这些软件元素称为软件构件。

一般情况下,在软件开发中采用重用软件构件,较从头开发这个软件更加容易。软件重用的目的是能更快、更好、成本更低地生产软件产品。软件的重用可划分为三个层次,即知识重用,方法和标准的重用以及软件成分的重用。

7.5.2 软件重用过程

通常先分析软件的功能需求,根据需求开发可重用的软件构件,并对其进行标识、构造、分类和存储,以便在特定的应用领域重用这些软件构件。在开发软件时,根据软件需要检索软件构件,对软件构件进行补充、修改、组装,再进行系统测试、调试工作,直到完成软件工程。

1. 软件重用过程模型

软件重用过程有三种模型,分别为组装模型、类重用模型和软件重用过程模型。

1) 组装模型

最简单的软件重用过程是,先将以往软件工程项目中建立的软件构件存储在构件库中,然后通过对软件构件库进行查询,提取可以重用的软件构件;再对它们进行一些修改,以适应新系统,并建造新系统需要的其他构件(这些构件可保存到构件库中,以便今后重用);最后将新系统需要的所有构件进行组装。

2) 类重用模型

利用面向对象技术,可以比较方便有效地实现软件重用。面向对象技术中的类是比较理想的可重用软件构件,不妨称之为类构件。

类构件的重用方式有以下几种:

(1) 实例重用。按照需要创建类的实例,然后向该实例发送适当的消息,启动相应的服务,完成所需要的工作。

(2) 继承重用。利用面向对象方法的继承性机制,子类可以继承父类已经定义的所有数据和操作,子类也可以定义新的数据和操作。

为了提高继承重用的效果,可以设计一个合理的、具有一定深度的类构件的层次结构。这样可以降低类构件的接口复杂性,提高类的可理解性,为软件开发人员提供更多的可重用构件。

(3) 多态重用。多态重用方法根据接收消息的对象类型不同,在响应一个一般化的消息时,由多态性机制启动正确的方法,执行不同的操作。

3) 软件重用过程模型

为了实现软件重用,已经出现了许多过程模型,这些模型都强调领域工程和软件工程同时进行。

领域工程在特定的领域创建应用领域的模型，设计软件体系结构模型，开发可重用的软件成分，建立可重用的软件构件库。显然，软件构件库应当不断积累、不断完善。

基于构件的软件工程，根据用户的实际需求，参照领域模型进行系统分析，使用领域的结构模型进行结构设计；从可重用软件构件库中查找所需要的构件，对构件进行鉴定、调整，构造新的软件构件；再对软件构件进行组合，开发应用软件，软件构件不断更新，并补充到可重用软件构件库中，即完成软件重用过程。

2. 开发可重用的软件构件

开发可重用的软件构件的过程就是领域工程。所谓“领域”，是指具有相似或相近的软件需求的应用系统所覆盖的一组功能区域。可以根据领域的特性及相似性预测软件构件的可重用性。一旦确认了软件构件的重用价值，对这样的软件构件进行设计和构造，形成可重用的构件，进而建立软件构件库的过程，就是领域工程。领域工程的目的是标识、构造、分类和传播软件构件，以便在特定的应用领域重用这些软件构件。

领域工程包括分析过程、开发软件构件和传播软件构件三个主要的活动。

1) 分析过程

领域工程分析过程中，标识可重用的软件构件是一项重要工作。对以下问题的回答可作为标识可重用构件的指南：

- (1) 构件的功能在未来的工作中需要吗？
- (2) 构件的功能通用性强吗？
- (3) 构件是否依赖于硬件？
- (4) 构件的设计是否足够优化？
- (5) 构件在重用时需要做大的修改吗？
- (6) 能否将一个不可重用的构件分解成一组可重用的构件？

2) 开发可重用的软件构件

为了开发可重用的软件构件，应该考虑以下几个问题：

(1) 标准的数据结构。例如，文件结构或数据库结构，所有构件都使用这些标准的数据结构。

(2) 标准的接口协议。建立模块内接口、外部接口和人机界面三个层次的接口协议。

(3) 程序模板。可用结构模型作为程序体系结构设计的模板。例如，报警系统软件可用于医疗监护、家庭安全、工业过程监控等各种系统，它的结构模板含有如下构件：

- 人机交互界面。
- 安全范围设置。
- 与监控传感器通信的管理机制。
- 响应机制。
- 控制机制。

建立了上述程序模板后，在设计具体的应用软件时，只要根据系统的实际要求稍加修改就可实现重用。

3) 传播软件构件

传播软件构件就是让用户能在成千上万的软件构件中找到自己所需要的构件,这需要很好地描述构件。构件的描述包括构件的功能、使用条件、接口、如何实现等。构件如何实现的问题,只有准备修改构件的人需要知道,其他人只要了解构件的功能、使用条件和接口即可。

3. 分类和检索软件构件

随着软件构件的不断丰富,软件构件库的规模不断扩大,软件构件库的组织结构将直接影响构件的检索效率。软件构件库结构的设计和检索方法的选用,应当尽量保证用户容易理解,易于使用。

对可重用软件构件库进行分类,可便于用户的检索使用。构件分类的方法有枚举分类、刻面分类和属性值分类三种典型模式。

4. 软件构件重用环境

软件构件的重用必须由相应的环境来支持,环境应包含下列元素:

- (1) 软件构件库。存放软件构件和检索构件所需要的分类信息。
- (2) 软件构件库管理系统。管理对构件库的访问。
- (3) 软件构件库检索系统。用户应用系统通过检索系统检索构件、重用构件。
- (4) CASE 工具。帮助用户把重用的构件集成到新的设计中去。

上述功能可以嵌入软件构件重用库中。重用库中存储各种各样的软件成分,例如,规格说明、设计、代码、测试用例及用户指南等。重用库包括有关构件的数据库以及数据库的查询工具,构件分类模式是构件数据库查询的基础。

7.5.3 软件逆向工程

1. 逆向工程概念

在工程技术人员的一般概念中,产品设计生产过程是一个从设计到完成产品的过程,即设计人员首先在大脑中构思产品的外形、性能和大致的技术参数等,然后在详细设计阶段完成各类数据模型,再将这个模型转入研发流程,最终完成产品的整个设计研发周期。这样的产品设计过程称为“正向设计”过程。

逆向工程(Reverse Engineering)是一种产品设计技术的再现过程,即对一项目标产品进行逆向分析及研究,从而演绎并得出该产品的处理流程、组织结构、功能特性及技术规格等设计要素,进而制作出功能相近,但又不完全一样的产品。简单地说,逆向工程就是根据已有的产品,反向推出产品设计数据(包括各类设计图或数据模型)的过程。因此,逆向工程可以被认为是一个从产品到设计的过程。

由于法律对知识产权的保护,复制制造与别人完全一样的产品是不允许的。因此逆向工程可能会被误认为是对知识产权的严重侵害,但是在实际应用中,反而可能会保护知识产权所有者。例如,在集成电路领域,如果怀疑某公司侵犯知识产权,可以用逆向工程技术来寻找证据。

逆向工程又名反向工程,源于商业及军事领域的硬件制造业。相互竞争的公司为了解对方设计

和制造工艺的机密，在得不到设计和制造说明书的情况下，通过拆解实物获得产品生产信息。软件的逆向工程基本类似，只不过通常拆解分析的不仅有竞争对手的程序，还有自己公司的软件。

软件的逆向工程是分析程序，力图在比源代码更高的抽象层次上建立程序的表示过程。它不仅是设计的恢复过程，还可以借助工具从已存在的程序中抽取数据结构、体系结构和程序设计信息。软件逆向工程也可被视作“开发周期的逆行”。对一项软件程序进行逆向工程，类似于逆行传统瀑布模型中的开发步骤，即根据实现阶段的输出（即软件程序）还原出在设计阶段所做的构思。软件逆向工程仅仅是一种检测或分析的过程，它并不会更改目标系统。软件逆向工程可以使用净室技术来避免侵犯版权。

2. 逆向工程实现方法

软件逆向工程有多种实现方法，主要包括以下三种：

（1）分析信息交换过程。最常用于协议逆向工程，涉及使用总线分析器和数据包嗅探器。在接入计算机总线或网络连接，并成功截取通信数据后，可以对总线或网络行为进行分析，以制造出拥有相同行为的通信实现。此法特别适用于设备驱动程序的逆向工程。

（2）反汇编。使用反汇编器，把程序的原始机器码翻译成较便于阅读理解的汇编代码。这适用于任何计算机程序，对不熟悉机器码的人特别有用，流行的相关工具有 OllyDebug 和 IDA。

（3）反编译。使用反编译器，尝试从程序的机器码或字节码重现高级语言形式的源代码。

随着计算机技术在各个领域的广泛应用，特别是软件开发技术的迅猛发展，基于某个软件，以反汇编阅读源码的方式去推断其数据结构、体系结构和程序设计信息，已成为软件逆向工程技术关注的主要方向。目前应用的主流逆向工程软件有 Imageware、Geomagic Studio、CopyCAD、RapidForm、UG。

软件逆向工程的目的是研究和学习先进的技术，特别是当手里没有合适的文档资料，又很需要实现某个软件功能的时候。也正因为这样，很多软件为了垄断技术，在软件安装之前，会要求用户同意不去逆向研究。

7.5.4 软件再工程

软件再工程是软件逆向工程的扩充。软件逆向工程提取需要的信息后，软件再工程基于对原系统的理解和转换，在不变更整个系统功能的前提下，产生新的软件源代码。软件再工程是以软件工程方法学为指导，对既有对象系统进行调查，并将其重构为新形式代码的开发过程，分为再分析、再编码、再测试三个阶段。

从软件重用方法学来说，如何开发可重用软件 and 如何构造采用可重用软件的系统体系结构是两个最关键的问题。不过对再工程来说，前者很大一部分内容是对既有系统中非可重用构件的改造。因此可以使用 CASE 工具（逆向工程和再工程工具）来帮助理解原有的设计。

软件再工程的成本取决于重做工程的难度。源代码转换最便宜，程序结构重构次之，程序和数据重构成本较高，体系结构迁移成本最高。尽管如此，软件再工程与重新开发软件相比，成本和风险还是更小一些，原因是软件工程的起点不同。在软件再工程的各个阶段，软件的可重用程度决定着软件再工程的工作量。

7.6 软件产品线

软件产品线（Software Product Line）是一个产品集合，这些产品共享一个公共的、可管理的特征集，这个特征集能满足特定领域的特定需求。软件产品线是一个十分适合专业开发组织的软件开发方法，能有效地提高软件生产率和质量，缩短开发时间，降低总开发成本。

软件产品线主要由两部分组成，分别是核心资源和产品集合。核心资源是领域工程的所有结果的集合，是产品线中产品构造的基础。核心资源必定包含产品线中所有产品共享的产品线架构，新设计开发的或者通过对现有系统的再工程得到的、需要在整个产品线中系统化复用的构件，与构件相关的测试计划、测试实例以及所有设计文档，需求说明书、领域模型、领域范围的定义，以及采用 COTS 的构件。

7.6.1 产品线的过程模型

软件产品线的过程模型主要有双生命周期模型、SEI 模型和三生命周期模型。

1. 双生命周期模型

双生命周期模型分成两个重叠的生命周期，分别是领域工程和应用工程，如图 7-6 所示。

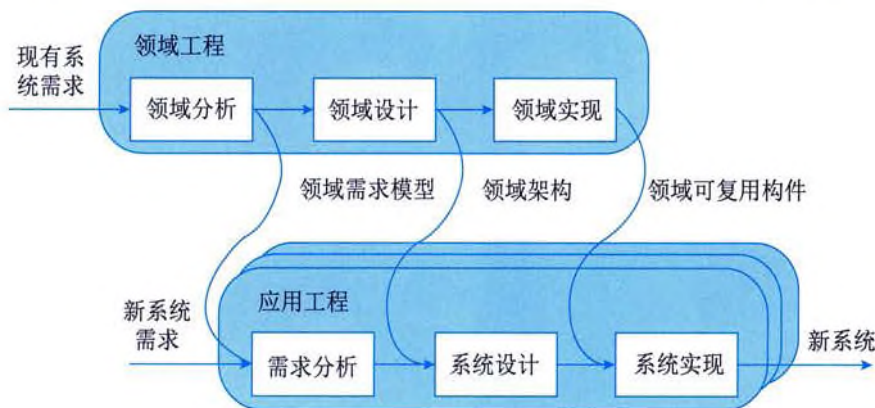


图 7-6 产品线的双生命周期模型

领域工程阶段的主要任务有：

- (1) 领域分析。利用现有系统的设计、架构和需求建立领域模型。
- (2) 领域设计。用领域模型确定领域 / 产品线的共性和可变性，为产品线设计架构。
- (3) 领域实现。基于领域架构开发领域可复用资源，例如，构件、文档和代码生成器等。

应用工程在领域工程结果的基础上构造新产品。应用工程需要根据每个应用独特的需求，经过以下阶段，生成新产品：

- (1) 需求分析。将系统需求与领域需求比较，划分成领域公共需求和独特需求两部分，得出系统需求规格说明书。
- (2) 系统设计。在领域架构基础上，结合系统独特需求，设计应用的软件架构。
- (3) 系统实现。遵照应用架构，用领域可复用资源实现领域公共需求，用定制开发的构件

满足系统独特需求，构建新的系统。

应用工程将产品线资源不能满足的需求返回给领域工程，以检验是否将之并入产品线的需求中。领域工程从应用工程中获得反馈或结合新产品的需求，进入又一次周期性发展，称此为产品线的演化。

双生命周期模型定义了典型的产品线开发过程的基本活动、各活动内容和结果以及产品线的演化方式。这种产品线方法综合了软件架构和软件复用的概念，在模型中定义了一个软件工程化的开发过程，目的是提高软件生产率、可靠性和质量，降低开发成本，缩短开发时间。

2. SEI 模型

SEI 将产品线的基本活动分为三个部分，分别是核心资源开发（即领域工程）、产品开发（即应用工程）和管理，如图 7-7 所示。从本质上来看，产品线开发包括核心资源库的开发和使用核心资源的产品开发，这两者都需要技术和组织的管理。核心资源的开发和产品开发可同时进行，也可交叉进行。例如，新产品的构建以核心资源库为基础，或者核心资源库可从已存在的系统中抽取。

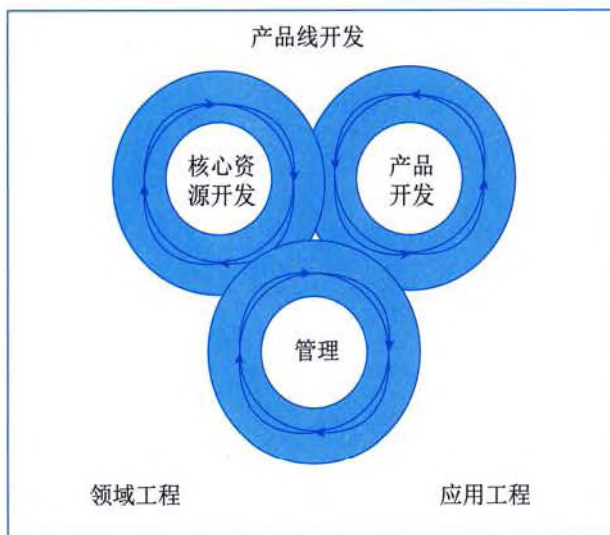


图 7-7 SEI 模型

在图 7-7 中，每个旋转环代表一个基本活动，三个环连接在一起，不停地运动着。三个基本活动交错连接，可以任何次序发生，且是高度重叠的。旋转的箭头表示不但核心资源库被用来开发产品，而且已存在的核心资源的修改，甚至新的核心资源常常可以来自产品开发。在核心资源和产品开发之间有一个强的反馈环，当新产品开发时，核心资源库就得到刷新。对核心资源的使用反过来又会促进核心资源的开发活动。另外，核心资源的价值通过使用它们的产品开发来得到体现。

SEI 模型的主要特点如下：

(1) 循环重复是产品线开发过程的特征，也是核心资源开发、产品开发，以及核心资源和产品之间协作的特征。

- (2) 核心资源开发和产品开发没有先后之分。
- (3) 管理活动协调整个产品线开发过程的各个活动，对产品线的成败负责。
- (4) 核心资源开发和产品开发是两个互动的过程，三个活动和整个产品线开发之间也是双向互动的。

3. 三生命周期模型

三生命周期模型是对双生命周期模型的一种改进，主要针对大型软件企业的软件产品线开发，如图 7-8 所示。三生命周期模型为有多个产品线的大型企业增加了企业工程（Enterprise Engineering）流程，以便在企业范围内对所有资源的创建、设计和复用提供合理的规划。为了强调产品线工程在满足市场需求上与一般的系统复用的区别，在领域工程中增加了产品线确认作为起始阶段，和领域分析阶段、架构开发阶段、基础设施开发阶段组成整个领域工程，还为领域分析阶段增加了市场分析的任务。同样，为应用工程增加了业务/市场分析。在领域工程和应用工程之间的双向交互中添加核心资源管理作为桥梁，核心资源管理和领域工程、应用工程之间的支持和交互是双向的，以便于产品线核心资源的管理和演化。

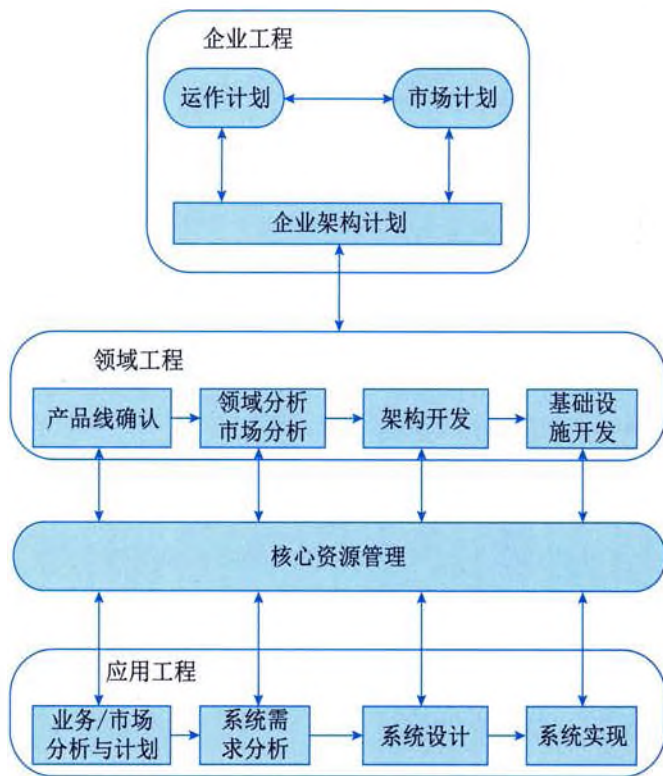


图 7-8 产品线的三生命周期模型

最后，需要说明的是，在上述三个过程模型中，软件产品线开发过程并没有明确描述如何复用遗留资源。实际上，大多数将要建立软件产品线的企业都积累了产品线所在领域的大量应用代码和相关文档，这些代码和文档中包含的知识对领域工程来说是至关重要的。

7.6.2 产品线的建立方式

软件产品线的建立需要企业有意识的、长期的努力才有可能成功。采用产品线方式开发软件时，开发人员可以划分为两组，分别是负责核心资源的小组和负责产品的小组。这也是产品线开发与独立系统开发的主要区别。

软件产品线的建立通常有4种方式，其划分依据有两个：第一个是企业采用演化方式（evolutionary）还是革命方式（revolutionary）引入产品线开发过程，第二个是企业基于现有产品开发还是开发全新的产品线。这4种方式的基本特征如表7-5所示。

表 7-5 软件产品线不同建立方式的基本特征

依据	演化方式	革命方式
基于现有产品	基于现有产品架构设计产品线的架构，经演化现有构件，开发产品线构件	核心资源的开发基于现有产品集的需求和可预测的、将来需求的超集
全新产品线	产品线核心资源随产品新成员的需求而演化	开发满足所有预期产品线成员的需求的核心资源

1. 将现有产品演化为产品线

在基于现有产品架构设计的产品线架构的基础上，将特定产品的构件逐步地、越来越多地转化为产品线的共用构件，从基于产品的方法“慢慢地”转化为基于产品线的软件开发。这种方法的主要优点是通过对投资回收期的分解、对现有系统演化的维持，使产品线方法的实施风险降到了最小，但完成产品线核心资源的总周期和总投资都比使用革命方式要大。

2. 用软件产品线替代现有产品集

基本停止现有产品的开发，所有工作直接针对软件产品线的核心资源开发。遗留系统只有在符合架构和构件需求的情况下，才可以和新的构件协作。这种方法的目标是开发一个不受现有产品集限制的、全新的平台，总周期和总投资较演化方式要少，但因重要需求的变化导致的初始投资报废的风险加大。另外，基于核心资源的第一个产品面世的时间将会推后。

除了投资和周期方面的考虑外，现有产品集中软硬件结合的紧密程度，以及不同产品在硬件方面的需求的差异，也是产品线开发采用演化还是革命方式的决策依据。对于软硬件结合密切，且硬件需求差异大的现有产品集，因无法满足产品线方法对软硬件同步的需求，只能采用革命方式替代现有产品集。

3. 全新软件产品线的演化

当企业进入一个全新的领域，要开发该领域的一系列产品时，同样也有演化和革命两种方式。演化方式将每一个新产品的需求与产品线核心资源进行协调。这种方式的好处是先期投资少，风险较小，第一个产品面世时间早。另外，因为是进入一个全新的领域，演化方式可以减少和简化因经验不足造成的初始阶段错误的修正代价。缺点是已有的产品线核心资源会影响新产品的需求协调，使成本加大。

4. 全新软件产品线的开发

设计师和开发工程师首先要得到产品线所有可能的需求,基于这个需求超集来设计和开发产品线核心资源。第一个产品将在产品线核心资源全部完成之后才开始构建。这种方式的优点是,一旦产品线核心资源完成后,新产品的开发速度将非常快,总成本也将减少;缺点是对新领域的需求很难做到全面和正确,使得核心资源不能像预期的那样支持新产品的开发。

从整体上来看,软件产品线的发展过程有三个阶段,分别是开发阶段、配置分发阶段和演化阶段。引起产品线架构演化的原因与引起任何其他系统演化的原因一样:产品线与技术变化的协调、现有问题的改正、新功能的增加、对现有功能的重组以允许更多的变化等。产品线的演化包括产品线核心资源的演化、产品的演化和产品的版本升级。

7.7 统一建模语言

统一建模语言(Unified Modeling Language, UML)是一种定义良好、易于表达、功能强大且普遍适用的建模语言,它融入了软件工程领域的新思想、新方法和新技术,它的作用域不限于支持面向对象分析和面向对象设计,还支持从需求分析开始的软件开发的全过程。

1. UML 的结构

从总体上来看,UML 的结构包括构造块、公共机制和规则三个部分。

(1) 构造块。UML 有三种基本的构造块,分别是事物(thing)、关系(relationship)和图(diagram)。事物是 UML 的重要组成部分,关系把事物紧密联系在一起,图是多个相互关联的事物的集合。

(2) 公共机制。公共机制是指达到特定目标的公共 UML 方法,主要包括规格说明(详细说明)、修饰、公共分类(通用划分)和扩展机制 4 种。规格说明是事物语义的细节描述,它是模型真正的核心;UML 为每个事物设置了一个简单的记号,还可以通过修饰来表达更多的信息;UML 包括两组公共分类,分别是类与对象(类表示概念,而对象表示具体的实体)、接口与实现(接口用来定义契约,而实现就是具体的内容);扩展机制包括约束(扩展了 UML 构造块的语义,允许增加新的规则或修改现有的规则)、构造型(扩展 UML 的词汇,用于定义新的构造块)和标记值(扩展了 UML 构造块的特性,允许创建新的特殊信息来扩展事物的规格说明)。

(3) 规则。规则是构造块如何放在一起的规定,具体包括:为构造块命名;赋予这些名称特定含义的上下文,即范围;怎样使用或看见名字,即可见性;事物如何正确、一致地相互联系,即完整性;运行或模拟动态模型的含义是什么,即执行。

UML 对系统架构的定义是系统的组织结构,包括系统分解的组成部分,以及它们的关联性、交互机制和指导原则等提供系统设计的信息。具体来说,就是指以下 5 个系统视图:

(1) 逻辑视图。逻辑视图也称为设计视图,它表示了设计模型中在架构方面具有重要意义的部分,即类、子系统、包和用例实现的子集。

(2) 进程视图。进程视图是可执行线程和进程作为活动类的建模,它是逻辑视图的一次执

行实例，描述了并发与同步结构。

(3) 实现视图。实现视图对组成基于系统的物理代码的文件和构件进行建模。

(4) 部署视图。部署视图把构件部署到一组物理节点上，表示软件到硬件的映射和分布结构。

(5) 用例视图。用例视图是最基本的需求分析模型。

另外，UML 还允许在一定的阶段隐藏模型的某些元素、遗漏某些元素，以及不保证模型的完整性，但模型要逐步地达到完整和一致。

2. 事物

UML 中的事物也称为建模元素，包括结构事物 (Structural Things)、行为事物 (Behavioral Things, 动作事物)、分组事物 (Grouping Things) 和注释事物 (Annotational Things, 注解事物)。这些事物是 UML 模型中最基本的 OO 构造块。

(1) 结构事物。结构事物在模型中属于最静态的部分，代表概念上或物理上的元素。UML 有 7 种结构事物，分别是类、接口、协作、用例、活动类、构件和节点。类是描述具有相同属性、方法、关系和语义的对象的集合，一个类实现一个或多个接口；接口是指类或构件提供特定服务的一组操作的集合，接口描述了类或构件的对外的可见的动作；协作定义了交互的操作，是一些角色和其他事物一起工作，提供一些合作的动作，这些动作比事物的总和要大；用例是描述一系列的动作，产生有价值的结果，在模型中用例通常用来组织行为事物，用例是通过协作来实现的；活动类的对象有一个或多个进程或线程，活动类和类很相似，只是它的对象代表的事物的行为和其他事物是同时存在的；构件是物理上或可替换的系统部分，它实现了一个接口集合；节点是一个物理元素，它在运行时存在，代表一个可计算的资源，通常占用一些内存且具有处理能力，一个构件集合一般来说位于一个节点，但有可能从一个节点转到另一个节点。

(2) 行为事物。行为事物是 UML 模型中的动态部分，代表时间和空间上的动作。UML 有两种主要的行为事物。第一种是交互 (内部活动)，交互是由一组对象之间在特定上下文中，为达到特定目的而进行的一系列消息交换而组成的动作。交互中组成动作的对象的每个操作都要详细列出，包括消息、动作次序 (消息产生的动作)、连接 (对象之间的连接)。第二种是状态机，状态机由一系列对象的状态组成。

(3) 分组事物。分组事物是 UML 模型中组织的部分，可以把它们看成是个盒子，模型可以在其中进行分解。UML 只有一种分组事物，称为包。包是一种将有组织的元素分组的机制。与构件不同的是，包纯粹是一种概念上的事物，只存在于开发阶段，而构件可以存在于系统运行阶段。

(4) 注释事物。注释事物是 UML 模型的解释部分。

3. 关系

UML 用关系把事物结合在一起，主要有下列 4 种关系：

(1) 依赖 (dependency)。依赖是两个事物之间的语义关系，其中一个事物发生变化会影响另一个事物的语义。

(2) 关联 (association)。关联描述一组对象之间连接的结构关系。

(3) 泛化 (generalization)。泛化是一般化和特殊化的关系, 描述特殊元素的对象可替换一般元素的对象。

(4) 实现 (realization)。实现是类之间的语义关系, 其中的一个类指定了由另一个类保证执行的契约。

4. 图

UML 2.0 包括 14 种图, 分别列举如下:

(1) 类图 (Class Diagram)。类图描述一组类、接口、协作和它们之间的关系。在 OO 系统的建模中, 最常见的图就是类图。类图给出了系统的静态设计视图, 活动类的类图给出了系统的静态进程视图。

(2) 对象图 (Object Diagram)。对象图描述一组对象及它们之间的关系。对象图描述了在类图中所建立的事物实例的静态快照。和类图一样, 这些图给出系统的静态设计视图或静态进程视图, 但它们是从真实案例或原型案例的角度建立的。

(3) 构件图 (Component Diagram)。构件图描述一个封装的类和它的接口、端口, 以及由内嵌的构件和连接件构成的内部结构。构件图用于表示系统的静态设计实现视图。对于由小的部件构建大的系统来说, 构件图是很重要的。构件图是类图的变体。

(4) 组合结构图 (Composite Structure Diagram)。组合结构图描述结构化类 (例如, 构件或类) 的内部结构, 包括结构化类与系统其余部分的交互点。组合结构图用于画出结构化类的内部内容。

(5) 用例图 (Use Case Diagram)。用例图描述一组用例、参与者及它们之间的关系。用例图给出系统的静态用例视图。这些图在对系统的行为进行组织和建模时是非常重要的。

(6) 顺序图 (Sequence Diagram, 也称为序列图)。顺序图是一种交互图 (Interaction Diagram), 交互图展现了一种交互, 它由一组对象或参与者以及它们之间可能发送的消息构成。交互图专注于系统的动态视图。顺序图是强调消息的时间次序的交互图。

(7) 通信图 (Communication Diagram)。通信图也是一种交互图, 它强调收发消息的对象或参与者的组织结构。顺序图和通信图表达了类似的基本概念, 但它们所强调的概念不同, 顺序图强调的是时序, 通信图强调的是对象之间的组织结构 (关系)。在 UML 1.X 版本中, 通信图称为协作图 (Collaboration Diagram)。

(8) 定时图 (Timing Diagram, 计时图)。定时图也是一种交互图, 它强调消息跨越不同对象或参与者的实际时间, 而不仅仅关心消息的相对顺序。

(9) 状态图 (State Diagram)。状态图描述一个状态机, 它由状态、转移、事件和活动组成。状态图给出了对象的动态视图。它对于接口、类或协作的行为建模尤为重要, 而且它强调事件导致的对象行为, 这非常有助于对反应式系统建模。

(10) 活动图 (Activity Diagram)。活动图将进程或其他计算结构展示为计算内部的控制流和数据流。活动图专注于系统的动态视图。它对系统的功能建模和业务流程建模特别重要, 并强调对象间的控制流程。

(11) 部署图 (Deployment Diagram)。部署图描述对运行时的处理节点及在其中生存的构

件的配置。部署图给出了架构的静态部署视图，通常一个节点包含一个或多个部署图。

(12) 制品图 (Artifact Diagram)。制品图描述计算机中一个系统的物理结构。制品包括文件、数据库和类似的物理比特集合。制品图通常与部署图一起使用。制品也给出了它们实现的类和构件。

(13) 包图 (Package Diagram)。包图描述由模型本身分解而成的组织单元，以及它们之间的依赖关系。

(14) 交互概览图 (Interaction Overview Diagram)。交互概览图是活动图和顺序图的混合物。

7.8 软件形式化方法

提高软件可靠性的一种重要技术是使用形式化方法。形式化方法是建立在严格数学基础上、具有精确数学语义的开发方法。广义的形式化方法是指软件开发过程中分析、设计和实现的系统工程方法，狭义的形式化方法是指软件规格和验证的方法。

1. 形式化方法概述

近年来，形式化方法已不再只是一种研究所里的学术研究工作，而是已经开始被工业界接受并应用于开发实际的系统。例如，在需求分析中，形式化方法的思想是利用形式化规格说明语言，严格定义用户需求，并采用数学推演的方法证明需求定义的性质，对于复杂的应用问题，尽管无法验证整个需求定义的完整性，但仍有可能为避免某些要点的疏漏而建立数学断言，然后予以形式证明或反驳。形式化规格说明语言包括严格的语法定义和语义定义，以及一系列的数学推演规则。这些规则不仅说明了某些数学性质在软件规格说明中是否成立，也说明了软件实现与软件规格说明之间的满足关系。

形式化方法的主要优越性在于它能够数学地表述和研究应用问题及其软件实现。但是，它要求开发人员具备良好的数学基础。用形式化语言书写的大型应用问题的软件规格说明往往过于细节化，并且难于为用户和软件设计人员所理解。由于这些缺陷，形式化方法在目前的软件开发实践中并未得到普遍应用。但是，近年来，形式化方法在以下两个方面的发展大大改善了其实用性：

(1) 形式化方法与图形语言机制相结合。为图形语言机制赋予形式化的语法和语义，从而兼具了图形表示的直观、简洁，以及形式化方法的严谨、精确等优点。

(2) 用 CASE (Computer Aided Software Engineering, 计算机辅助软件工程) 工具支持形式化软件开发。CASE 工具不仅可以简化描述工作，而且可以利用自动证明技术，帮助开发人员验证软件的数学性质。

实践证明，上述技术途径对于克服形式化方法的主要缺陷是行之有效的，因此，它们将在形式化方法的未来发展中发挥重要作用。

2. 净室软件工程

净室软件工程 (Cleanroom Software Engineering, CSE) 是软件开发的一种形式化方法，可

以开发较高质量的软件。它使用盒结构归约进行分析和建模,并且将正确性验证作为发现和排除错误的主要机制,使用统计测试来获取认证软件可靠性所需要的信息。

CSE 强调在规约和设计上的严格性,以及使用基于数学的正确性证明来对设计模型的每个元素进行形式化验证。作为对形式化方法的扩展,CSE 还强调统计质量控制技术,包括基于客户对软件的预期使用的测试。

CSE 的理论基础是函数理论和抽样理论,所采用的技术手段主要有以下 4 个方面:

(1) 统计过程控制下的增量式开发。

(2) 基于函数的规范、设计。CSE 按照函数理论定义了三种抽象层次,分别是行为视图、有限状态机视图和过程视图。规范从一个外部行为视图(称为黑盒)开始,然后被转化为一个状态机视图(称为状态盒),最后由一个过程视图(明盒)来实现。盒结构是基于对象的,并支持软件工程的关键原则,即信息隐藏、接口与实现分离。

(3) 正确性验证。正确性验证是 CSE 的核心,正是由于采用了这一技术,软件质量才有了极大的提高。

(4) 统计测试和软件认证。CSE 在测试方面采用统计学的基本原理,即当总体太大时必须采取抽样的方法。首先,确定一个使用模型来代表系统所有可能使用的(一般是无限的)总体;然后,由使用模型产生测试用例,因为测试用例是总体的一个随机样本,所以可得到系统预期操作性能的有效统计推导。

3. 程序验证

在软件工程领域,用于确保程序正确性的手段有很多,大体可分为两类:动态方法(Dynamic)和静态方法(Static)。动态方法需要将软件源码编译为可执行文件,再运行软件,通过特定的输入考察软件的运行结果。动态方法中使用得比较广泛的是软件测试,有手动测试、自动化测试等。动态方法效率高,错误定位准确,但应用场合受限,它要求必须具有程序能运行起来的环境。而且,其最大的问题在于,动态方法不能覆盖程序的所有输入和全部的执行路径,故只能证明在已有的输入下,被测试所覆盖的部分程序没有问题,而不能证明整个程序不存在错误。静态方法则是基于程序源代码进行分析,无须编译运行,所以应用场景更加广泛。常用的静态方法有静态分析、符号执行、定理证明、模型检测等。不同的静态方法各有优劣,其应用门槛也比较高,一般用于对程序正确性要求较高的场景。

(1) 静态分析(Static Analysis)方法比较轻量级,适用于大规模代码,但比较大的问题是较高比例的误报(False Positive)。所以静态分析报出的结果需要逐一人工排查,从而导致耗费大量的人力资源。比较有名的静态分析引擎如 Coverity,目前已经实现较大规模商业化应用。

(2) 符号执行(Symbolic Execution)可以看作是更加准确的测试方法,它通过符号值来静态执行程序,积累路径条件(Path Condition),直到到达目标位置,再对路径条件进行约束求解(Constraint Solving),判断目标位置的可达性(Reachability)。由于需要使用约束求解,而且对循环不友好,所以符号执行方法比较难以大规模应用。经典的符号执行工具如 KLEE,已经被加入 LLVM 的官方项目列表中。

(3) 定理证明(Theorem Proving)方法是使用高阶逻辑(High Order Logic, HOL)对程

序及其需要满足的性质进行建模描述，然后使用机器辅助证明的方法，一步一步证明程序能够满足要求的性质。定理证明方法的主要缺陷是自动化程度较低，需要大量的专业人力参与，编写证明代码，对软件的快速更新迭代不友好。辅助定理证明的典型工具是 Coq，于 2014 年获得 ACM 软件系统奖。此外，值得一提的是，基于定理证明方法验证的 C 语言编译工具链 CompCert，于 2021 年获得 ACM 软件系统奖。

（4）模型检测（Model Checking）是一种经典的形式化分析方法。它通过构造软件系统的抽象模型，来检测其是否满足要求的性质。模型检测方法的缺点是系统模型的建立需要领域专家的参与，寻找恰当的抽象层次，从而足以证明系统的特定属性是模型检测的一大难点。过分的抽象将导致属性无法证明，而不足的抽象又将导致太多属性无关的冗余细节，从而引发状态爆炸，无法在合理的时间内得到结果。经典的模型检测工具有 NuSMV、SPIN 等。其中，SPIN 于 2002 年获得 ACM 软件系统奖。此外，还有基于模型驱动开发（Model Driven Development）的 B 方法，以及其对应的工具链 Event B。